

Assignment 2 R

Fabian Lindh

December 2025

1 Task 1

1.1 OLS estimator

We begin by writing the code for the $\hat{B}_{OLS}$ estimator.

```
1 olsFun <- function(data){
2   y <- as.matrix(data[, 1]) # Dependent variable from the first column
3   x <- data[, 2] # Chooses the independent variable from the second column
4   X <- as.matrix(cbind(1, x)) # Adds column of ones to the matrix to estimate
5   #intercept
6
7   #We calculate XtX
8
9   XtX <- t(X) %*% X
10
11  #We calculate inverse of XtX
12  XtX_inv <- solve(XtX)
13
14  #We calculate XtY
15  XtY <- t(X) %*% y
16
17  #We calculate beta_ols
18  beta_ols <- XtX_inv %*% XtY
19
20  return(beta_ols[2]) #We return the coefficient
21 }
```

1.2 WLS estimation

We then continue with the $\hat{B}_{WLS}$ function

```
1 wlsFun <- function(data, lambda){
2   y <- as.matrix(data[, 1]) # Dependent variable, same as last function
3   x <- data[, 2] # Independent variable, same as last function
4   X <- as.matrix(cbind(1, x)) # Intercept, same as last function
5
6   #Lambda has to be numeric, else stop
7   if(!is.numeric(lambda)){
8     stop("lambda has to be a numeric value")
9   }
10
11  #Variance for each observation
12  variance_i <- exp(x * lambda)
13
14  #Create diagonal weight matrix W (Omega^-1) Since omega is the diagonal
15  #the inverse is:
16  W <- diag(1 / variance_i)
17
18  #X' W X
19  XtWX <- t(X) %*% W %*% X
```

```

20
21 #Inverse of X' W X
22 XtWX_inv <- solve(XtWX)
23
24 #X' W y
25 XtWy <- t(X) %*% W %*% y
26
27 #Finally we calculate the WLS estimator
28 beta_wls <- XtWX_inv %*% XtWy
29
30 #Return the coefficient
31 return(beta_wls[2])
32
33 }

```

1.3 FWLS estimation

Then finally we write the $\hat{B}_{FWLS}$ function

```

1 fwlFun <- function(data, trueVar){
2   y <- data[, 1] # Dependent variable
3   x <- data[, 2] # Independent variable
4   X <- as.matrix(cbind(1, x)) # Same as last function
5
6   #OLS estimation
7   XtX_inv <- solve(t(X) %*% X)
8   beta_ols <- XtX_inv %*% t(X) %*% y
9
10  #Residuals
11  u_hat <- y - X %*% beta_ols
12  u_hat_sq <- u_hat^2
13
14  #Lambda estimation
15
16  X_lambda <- as.matrix(cbind(1, x))
17  ln_u_hat_sq <- log(u_hat_sq)
18
19  #Beta lambda
20  beta_lambda <- solve(t(X_lambda) %*% X_lambda) %*% t(X_lambda) %*% ln_u_hat_sq
21  lambda_hat <- beta_lambda[2]
22
23  #Lambda has to be numeric, else stop
24  if(!is.numeric(lambda_hat)){
25    stop("lambda_hat has to be a numeric value")
26  }
27
28  #Calculating variance with the correct and false form
29  if(trueVar == TRUE){
30    variance_i <- exp(x * lambda_hat)
31  } else {
32    variance_i <- (1 + x * lambda_hat)
33  }
34  #Omega inverse
35  W <- diag(1 / variance_i)
36
37  #FWLS estimator
38  XtWX <- t(X) %*% W %*% X
39  XtWX_inv <- solve(XtWX)
40  XtWy <- t(X) %*% W %*% y
41
42  #Beta FWLS
43  beta_fwls <- XtWX_inv %*% XtWy
44

```

```

45 #Return results
46 return(beta_fwls[2])
47 }

```

1.4 Simulation

To check that the code is working correctly, we use the example data to check the output. After ensuring the functions generate the correct data we continue with the data generating function with the following parameters suitable for simple linear regression:

$$\begin{aligned}
 x_i &\sim U(0, 2) \\
 \epsilon &\sim N(0, e^{x_i\lambda}) \\
 \beta &= 2 \\
 y_i &= \beta x_i + \epsilon_i
 \end{aligned}$$

```

1 DataFun <- function(n, lambda) {
2   #x_i ~ U(0, 2)
3   x <- runif(n, min = 0, max = 2)
4   #Heteroskedastic standard deviation
5   sigma_i <- sqrt(exp(lambda * x))
6   #Error term u_i
7   epsilon <- rnorm(n, mean = 0, sd = sigma_i)
8   #Beta set to 2
9   beta <- 2
10  #y_i
11  y_i <- beta * x + epsilon
12  #Combine y and x to data matrix
13  data <- cbind(y_i, x)
14  return(data)
15 }

```

Finally we run the simulation function:

```

1 SimFun <- function(n, sim_reps, seed, lambda) {
2   # Set seed
3   set.seed(seed)
4
5   #Storage for estimates
6   ols_est <- numeric(sim_reps)
7   wls_est <- numeric(sim_reps)
8   fwls_true_est <- numeric(sim_reps)
9   fwls_false_est <- numeric(sim_reps)
10
11  #Simulation loop that uses data from DataFun
12  for (i in 1:sim_reps) {
13    data <- DataFun(n, lambda)
14    ols_est[i] <- olsFun(data = data)
15    wls_est[i] <- wlsFun(data = data, lambda = lambda)
16    fwls_true_est[i] <- fwlsFun(data = data, trueVar = TRUE)
17    fwls_false_est[i] <- fwlsFun(data = data, trueVar = FALSE)
18  }
19
20
21  #We extract the variances of the estimates from the simulation
22  var_results <- c(var_ols = var(ols_est),
23    var_wls = var(wls_est),
24    var_fwls_true = var(fwls_true_est),
25    var_fwls_false = var(fwls_false_est)
26  )
27
28  return(var_results)

```

29 }

We run the simulation and gather the results for plotting with the following code:

```
1 #Run the simulations and store the results
2 simresult_25 <- SimFun(n = 25, sim_reps = 1000, seed = 2024, lambda = 2)
3 simresult_50 <- SimFun(n = 50, sim_reps = 1000, seed = 2024, lambda = 2)
4 simresult_100 <- SimFun(n = 100, sim_reps = 1000, seed = 2024, lambda = 2)
5 simresult_200 <- SimFun(n = 200, sim_reps = 1000, seed = 2024, lambda = 2)
6 simresult_400 <- SimFun(n = 400, sim_reps = 1000, seed = 2024, lambda = 2)
7
8 #Combine the results from simulation in a dataframe for easier plotting
9 results_df <- data.frame(N = c(25, 50, 100, 200, 400),
10   var_OLS = c(simresult_25[1],
11   simresult_50[1],
12   simresult_100[1],
13   simresult_200[1],
14   simresult_400[1]),
15   var_WLS = c(simresult_25[2],
16   simresult_50[2],
17   simresult_100[2],
18   simresult_200[2],
19   simresult_400[2]),
20   var_FWLS_True = c(simresult_25[3],
21   simresult_50[3],
22   simresult_100[3],
23   simresult_200[3],
24   simresult_400[3]),
25   var_FWLS_False = c(simresult_25[4],
26   simresult_50[4],
27   simresult_100[4],
28   simresult_200[4],
29   simresult_400[4]))
30
31 library(tidyr)
32 library(ggplot2)
33
34 # Reshape data for plotting a line plot with ggplot format
35 #we want a long format, all columns except N to be gathered
36 plot_data <- results_df %>%
37   pivot_longer(cols = starts_with("var_"), #Select all
38     #columns starting with "var_"
39     names_to = "Estimator", #New column for estimator names
40     values_to = "Variance") #New column for variance values
41
42 #Finally we plot the data
43 ggplot(plot_data, aes(x = N, y = Variance, color = Estimator)) +
44   geom_line() + # Draws line between the points
45   geom_point() + #Dots at each variance estimate for each N
46   scale_x_continuous(breaks = c(25, 50, 100, 200, 400)) + #Breaks for the
47   #simulation sizes
48   labs(x = "Sample Size (N)",
49     y = "Variance Estimate") +
50   theme_minimal()
```

1.5 Simulation results

Running the simulation for sample sizes $n \in (25, 50, 100, 200, 400)$ gives the following results:

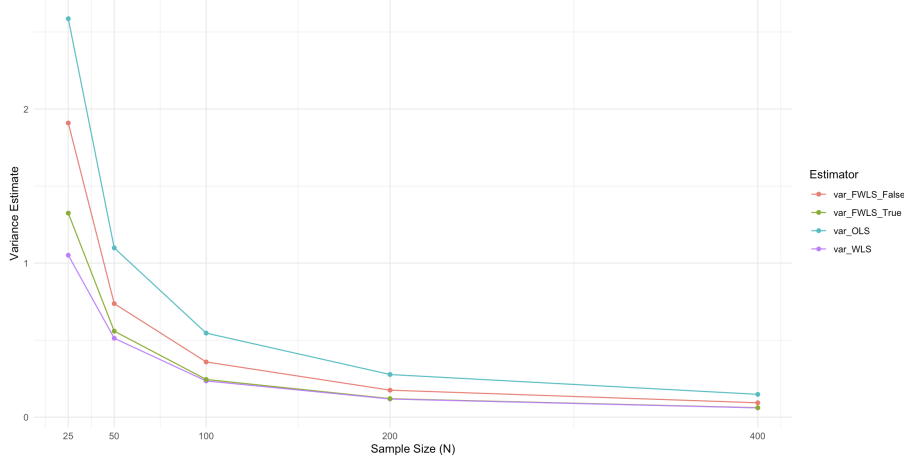


Figure 1: Variance Estimates by Estimator and Sample Size

The simulation has estimated the regression coefficient (β_1) using four different estimators, OLS, WLS, and FWLS using the true variance form and the false variance form. To test and compare their efficiency, the empirical variance was calculated from 1000 repetitions for five different sample sizes. Since the heteroskedasticity is deliberately modeled into the data used, since the variance of the error term from datafun is defined as $\sigma_i^2 = e^{x_i\lambda}$, we can evaluate the efficiency of each estimator in handling the non-constant variance.

From the plot, it can clearly be seen that the WLS estimator was more efficient than OLS under heteroskedasticity, as expected from the theory. This estimator, along with FWLS true, were the most efficient estimators for all sample sizes with the lowest variance. These two estimators are almost identical in performance, where WLS assumes that λ is known and $FWLS_{true}$ assumes the true form of the heteroskedasticity. λ acts as the growth rate for the variance in the data. Since FWLS tries to estimate using the true form of the variance, the estimator will slowly converge towards the efficiency of the WLS estimator when the sample size grows. It therefore makes sense that these two estimators are very similar and slowly become the same line as the sample size increases.

It can also be seen in the plot that estimating the variance with the erroneous form $1 + x_i\lambda$, assuming that the variance is linear, results in lower efficiency; however, it still managed to outperform the OLS estimator. This suggests that correcting for heteroskedasticity with the misspecified form for the data used is still more effective than not trying to correct for it at all. Even if efficiency is lost when comparing with the FWLS true form, FWLS with the false form still managed to achieve lower variance than the OLS estimator.

2 Task 2

2.1

We begin with visualizing the data galaxies, which include galaxy velocities. A density plot is constructed to get a visual impression of the data's distribution and potential clusters. The code uses ggplot for extra tidiness, and the labels are set for clear interpretation, and the limits are calibrated since the first plot did not include all data in the plot.

```
1 library(ggplot2)
2 galaxies
3 #TASK 1#####
4 #Create data frame for plotting the data
5 galaxies_df <- data.frame(Velocity = galaxies)
6 #Create density plot
7 ggplot(galaxies_df, aes(x = Velocity)) +
8   geom_density(color = "blue", fill = "lightblue") +
9   labs(x = "Velocity (km/sec / 1000)",
10        y = "Density") + xlim(7,37) +
11   theme_minimal()
```

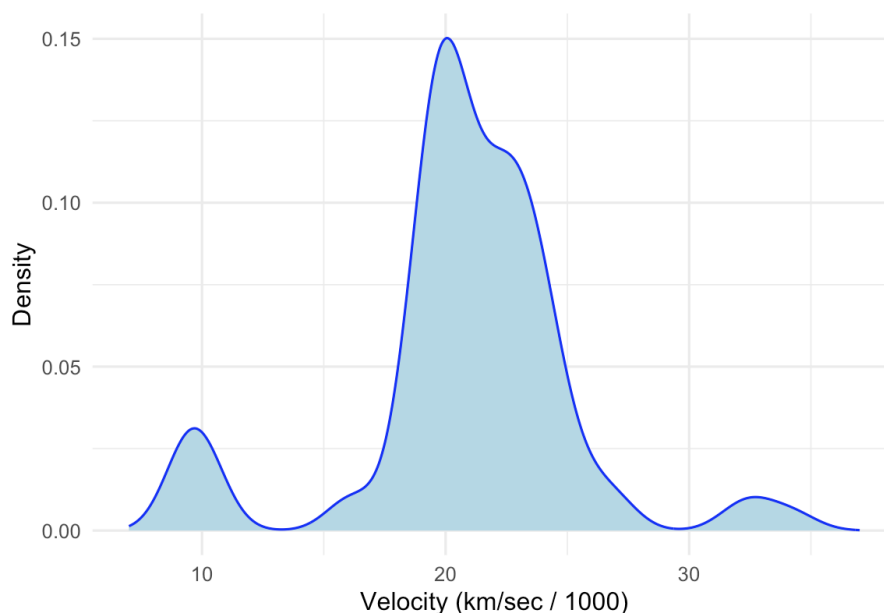


Figure 2: Density for galaxy velocities

Based on the density plot, there is no clear answer to how many components, but the most plausible seems to be three or maybe four natural components to sufficiently describe the data.

2.2

2.2.1 a)

The first step in implementing a mixture model is the E-step. This step involves calculating the posterior probabilities, $\gamma(z_{nk})$, which represent the responsibility that component k takes for generating the observation x_n .

The formula for the posterior probability is given by Equation (7):

$$\gamma(z_{nk}) = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \sigma_k^2)}{\sum_j \pi_j \mathcal{N}(x_n | \mu_j, \sigma_j^2)}$$

The code iterates through every data point and every component to calculate the normalized probability, $\gamma(z_{nk})$, that each point belongs to that specific component based on the current parameter estimates. The resulting matrix defines the responsibility distribution used to update the model in the next step.

```

1 #Task 2 a) #####
2 #Implement E-steps update function (Equation 7)
3 gammaUpdate <- function(x, mu, sigma, pi) {
4   #Initialize the length of x and mu
5   N <- length(x)
6   K <- length(mu)
7
8   # Initialize matrices and vectors
9   density_matrix <- matrix(0, nrow = N, ncol = K)
10  numerator <- matrix(0, nrow = N, ncol = K)
11  denominator_vector <- rep(0, N)
12  gamma_matrix <- matrix(0, nrow = N, ncol = K)
13
14  #Calculate the density values
15  for (n in 1:N) {
16    for (k in 1:K) {
17      density_matrix[n, k] <- dnorm(x[n], mean = mu[k], sd = sigma[k])
18    }
19  }
20  #Calculate the numarotor
21  for (n in 1:N) {
22    for (k in 1:K) {
23      numerator[n, k] <- pi[k] * density_matrix[n, k]
24    }
25  }
26  #Calculate the denominator
27  for (n in 1:N) {
28    denom_sum <- 0
29    for (k in 1:K) {
30      denom_sum <- denom_sum + numerator[n, k]
31    }
32    denominator_vector[n] <- denom_sum
33  }
34  #Compute gamma (i.e numarotor / denominator)
35  for (n in 1:N) {
36    for (k in 1:K) {
37      gamma_matrix[n, k] <- numerator[n, k] / denominator_vector[n]
38    }
39  }
40  #Return the gamma matrix
41  return(gamma_matrix)
42 }

```

2.2.2 b)

The next step is to implement the M-step given by equation 8, square root of equation 9 and equation 10.

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n, \quad k = 1, \dots, K$$

$$(\sigma_k)^{\text{new}} = \sqrt{\frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k^{\text{new}})^2}, \quad k = 1, \dots, K$$

$$\pi_k^{\text{new}} = \frac{N_k}{N}, \quad k = 1, \dots, K$$

This code updates the model's parameters: means (μ), standard deviations (σ), and mixture proportions (π). It uses the responsibilities (γ) from the code in Task 2a) as weights for the data. By performing these weighted summations, the algorithm determines the optimal new parameters that maximize the log-likelihood of the data given the current assignments.

```

1      #Task 2 b) #####
2      #Implement M-steps update functions (i.e equation 8, 9 & 10)
3      #Function to calculate N_k to simplify the M-step functions
4      calculate_Nk <- function(gamma) {
5          N_k <- rep(0, ncol(gamma))
6          for (k in 1:ncol(gamma)) {
7              N_k[k] <- sum(gamma[, k])
8          }
9          return(N_k)
10     }
11     #Equation 8, muUpdate (Update the means))
12     muUpdate <- function(x, gamma, sigma) {
13         N_k <- calculate_Nk(gamma)
14         K <- ncol(gamma)
15         mu_new <- rep(0, K)
16         #Update mu for each component
17         for (k in 1:K) {
18             sum_weighted <- 0
19             for (n in 1:length(x)) {
20                 sum_weighted <- sum_weighted + gamma[n, k] * x[n]
21             }
22             mu_new[k] <- sum_weighted / N_k[k]
23         }
24         #Return the updated means
25         return(mu_new)
26     }
27     #Equation 9, sigmaUpdate (Update the standard deviations)
28     sigmaUpdate <- function(x, gamma, mu) {
29         N_k <- calculate_Nk(gamma)
30         K <- length(mu)
31         N <- length(x)
32         sigma_new <- rep(0, K)
33         #Update sigma for each component
34         for (k in 1:K) {
35             sum_weighted_squared <- 0
36             for (n in 1:N) {
37                 diff <- x[n] - mu[k]
38                 sum_weighted_squared <- sum_weighted_squared + gamma[n, k] * (diff^2)
39             }
40             variance_new <- sum_weighted_squared / N_k[k]
41             sigma_new[k] <- sqrt(variance_new)
42         }
43         #Return the updated standard deviations
44         return(sigma_new)
45     }
46     #Equation 10, piUpdate (Update the mixture proportions)
47     piUpdate <- function(gamma) {
48         N <- nrow(gamma) #Number of data points
49         K <- ncol(gamma) #Number of components
50         N_k <- calculate_Nk(gamma) #Calculate N_k
51         pi_new <- rep(0, K) #Initialize new pi vector
52         #Update pi for each component
53         for (k in 1:K) {
54             pi_new[k] <- N_k[k] / N
55         }
56         #Return the updated mixture proportions
57         return(pi_new)
58     }

```

Our function is then tested for the given code in the Assignment, which gives the same output as the one demonstrated in the Assignment, which confirms that the code works

2.3

The next function calculates equation 6 which is given by

$$\ell = \log p(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\sigma}) = \sum_{n=1}^N \ln \left[\sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \sigma_k^2) \right]$$

The function calculates the total Log-Likelihood of the observed data given the mixture model parameters. It iterates through every data point, calculating its marginal probability by summing the weighted density contributions from all components. The function then returns the sum of the natural logarithm of these marginal probabilities, providing a single metric that quantifies the overall fitness of the model.

```

1      #Task 3 #####
2 #Implement log-likelihood function (Equation 6)
3 loglik <- function(x, pi, mu, sigma) {
4     n <- length(x)
5     K <- length(pi)
6     #Initialize log-likelihood
7     log_likelihood <- 0
8     #Calculate log-likelihood
9     for (i in 1:n) {
10
11         marginal_density <- 0
12         for (j in 1:K) {
13             density_value <- dnorm(x[i], mean = mu[j], sd = sigma[j])
14             marginal_density <- marginal_density + pi[j] * density_value
15         }
16         log_likelihood <- log_likelihood + log(marginal_density)
17     } #Return log-likelihood
18     return(log_likelihood)
19 }

```

2.4

The next step is to generate candidates that maximize the log likelihood. This code is given in the Assignment and is copied to complete the task. Then the EM function is finally implemented. The EM loop finds the mixture model parameters that best fit the data using the previous functions. This process iterates until the difference between the current log-likelihood and the new one is less than the default value 0.001, or 1000 iterations have been reached. This step is to prevent an infinite loop.

```

1      #Implement the function that runs the EM algorithm (This code is copied from the assignment)
2 EM <- function(x, K, tol = 0.001){
3     inits <- initialValues(x, K, reps = 1000)
4     mu <- inits$mu
5     sigma <- inits$sigma
6     prob <- inits$p
7     #Initial log-likelihood
8     logLik_old <- loglik(x, prob, mu, sigma)
9     logLik_new <- logLik_old #Initialize new log-likelihood for convergence check
10    iter <- 0
11    max_iter <- 1000 #Set the maximum number of iterations to prevent infinite loops
12
13    # EM-loop
14    repeat {
15        iter <- iter + 1
16        # E-step, calculate responsibilities
17        gamma <- gammaUpdate(x, mu, sigma, prob)
18
19        # M-step, update parameters
20        mu_new <- muUpdate(x, gamma, sigma)
21        sigma_new <- sigmaUpdate(x, gamma, mu_new)
22        prob_new <- piUpdate(gamma)

```

```

23     #Update each parameter
24     mu <- mu_new
25     sigma <- sigma_new
26     prob <- prob_new
27
28     #Calculate new log-likelihood
29     logLik_new <- loglik(x, prob, mu, sigma)
30
31     #Check for convergence if one of the conditions is met, break the loop
32     if (abs(logLik_new - logLik_old) < tol || iter >= max_iter) {
33         break
34     }
35
36     #Update old log-likelihood for next iteration
37     logLik_old <- logLik_new
38 }
39
40 return(list('loglik' = logLik_new, 'mu' = mu, 'sigma' = sigma, 'prob' = prob))
41 }
42

```

2.5

The last task is to run the EM-algorithm for $K = 2, 3, 4, 5$ to estimate the parameters using the galaxy data, and compare the log likelihoods, and lastly compare them in a plot. The code is running through the different values of K . The log-likelihood values are then saved in a data frame and printed for comparison. The last part is plotting, the different models are saved and plotted in the same plot. The reason why the plot is plotted like a grid is because of easier comparison, since the densities share the same shape in some areas, the difference between them and the original density was hard to compare.

```

1  # Task 5 #####
2  #The seed that was used in the assignment
3  set.seed(2024)
4  #Run the EM algorithm for K = 2 to 5
5  K_values <- 2:5
6  results <- list() #To store results for each K
7  for (K in K_values) {
8      results[[as.character(K)]] <- EM(galaxies, K = K, tol = 0.001)
9  }
10 # Print log-likelihoods for each K
11 loglik_values <- rep(0, length(K_values))
12 for (i in 1:length(K_values)) {
13     K <- K_values[i]
14     loglik_values[i] <- results[[as.character(K)]]$loglik
15 }
16 # Create a data frame to compare log-likelihoods
17 comparison_df <- data.frame(
18     K = K_values,
19     LogLikelihood = loglik_values)
20 #Print the comparison data frame
21 print(comparison_df)
22
23 #Plots #####
24 task1_density <- density(galaxies, from = 7, to = 37)
25 task1_df <- data.frame(x = task1_density$x, y = task1_density$y, source = "Original Density")
26 #Create an empty list to store plot data for each K
27 plot_list <- list()
28 #Generate density data for each K and combine with original density
29 for (K in 2:5) {
30     res <- results[[as.character(K)]]
31
32

```

```

33 k_dens <- rep(0, length(task1_df$x))
34
35 for (k in 1:K) {
36
37   k_dens <- k_dens + res$prob[k] * dnorm(task1_df$x, res$mu[k], res$sigma[k])
38 }
39 #Create data frame for each density
40 k_dens_df <- data.frame(
41   x = task1_df$x,
42   y = k_dens,
43   source = paste("Density, K =", K)
44 )
45 #Combine with original density
46 combined_df <- rbind(task1_df, k_dens_df)
47 combined_df$panel <- paste("K =", K)
48 #Store in the list
49 plot_list[[as.character(K)]] <- combined_df
50 }
51 #Combine all plot data into one data frame
52 all_plot_data <- do.call(rbind, plot_list)
53 ggplot(all_plot_data, aes(x = x, y = y, color = source)) +
54   geom_line(size = 0.8) +
55   facet_wrap(~ panel, ncol = 2) +
56   labs(
57     x = "Velocity (km/sec / 1000)",
58     y = "Density"
59   ) +
60   xlim(7, 37) +
61   ylim(0, 0.27) +
62   theme_minimal() +
63   scale_color_manual(values = c("Original Density" = "blue",
64     "Density, K = 2" = "red",
65     "Density, K = 3" = "green",
66     "Density, K = 4" = "purple",
67     "Density, K = 5" = "orange"))

```

2.6 Results/Conclusion

K	2	3	4	5
Log-Likelihood	-220.06	-217.93	-197.46	-195.98

Table 1: Log-Likelihood for mixture models

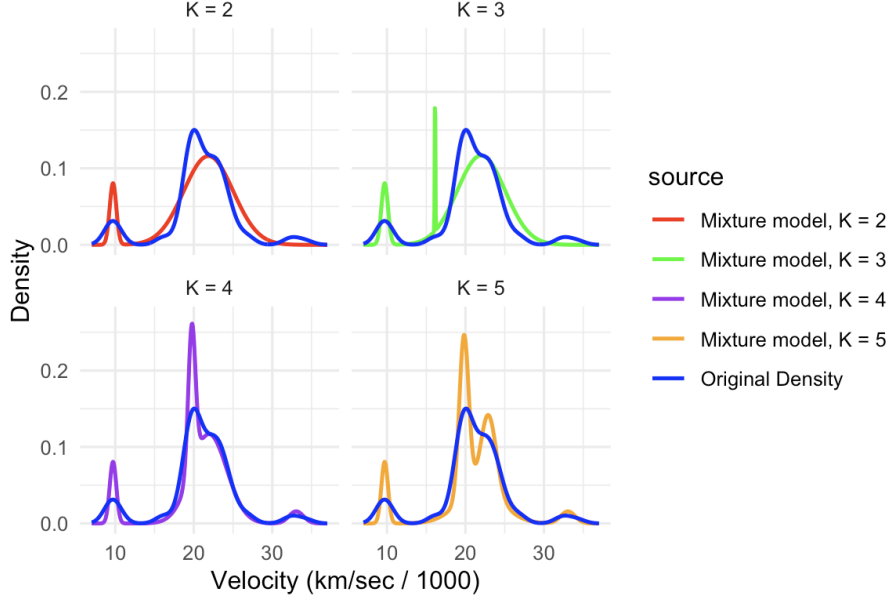


Figure 3: Comparison of mixture models densities

The mixture model that has the highest log-likelihood, i.e., fits the data the best, is when $K = 5$, which is not what was first thought when observing the data. We can see that the difference between log-likelihoods is relatively small between when $K = 4$ and when $K = 5$. For the visual comparison, we can see that none of the mixture models is actually a great fit. When $K = 3$ seems to have found a cluster that was not where we found it naturally. The EM algorithm may not work to its potential. Furthermore, selecting K based purely on maximum log-likelihood without penalizing model complexity introduces a risk of overfitting, as the model may begin to fit noise rather than the underlying data structure. Given the marginal gain in log-likelihood, other metrics should be consulted to more robustly determine the optimal number of components. Upon visual inspection of the resulting component distributions, it is apparent that none of the tested mixture models provide a strongly convincing fit, suggesting the EM algorithm may have converged to a local maximum rather than the global optimum.